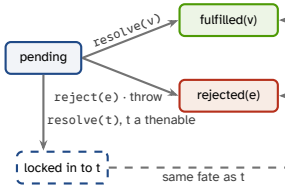


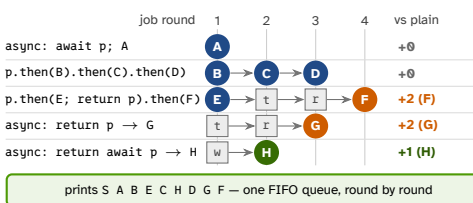
In one line: A promise is a **settle-once** result: its executor runs **now**, its handlers **always later**, each as a promise **job** (a microtask on the web). then returns a **new** promise that adopts whatever its handler returns; async/await runs on the same jobs. The work starts when it is called and a promise cannot be cancelled — an AbortSignal asks the work to stop.

1 states and fates



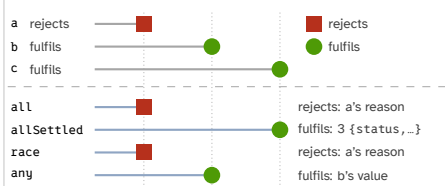
settled = fulfilled or rejected
resolved = settled or locked in: from then on resolve, reject and a throw are ignored
resolve(itself) → rejected with **TypeError**

2 counting jobs (code: left column)



t: job calling p.then: r resolves the outer promise · w: await resumes
returning a native promise: 2 extra jobs; return await p: 1 fewer
 await on a native promise or a plain value (await i): 1 job

3 combinators, same inputs



no rejection: all fulfils at t3 with [a,b,c], race at t1
 all reject: any → AggregateError, .errors in input order
nothing is cancelled: b and c run on after all rejects

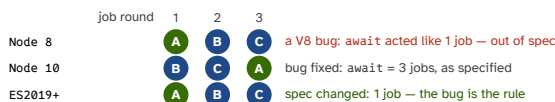
Why a promise behaves as it does

- Always async, on purpose.** The executor runs synchronously, but then never calls back synchronously — even on a settled promise the handler waits for a job. An API that is sometimes sync and sometimes async is impossible to reason about: the 2013 “do not release Zalgo” rule. Promise.try(f) (ES2025) closes the gap: a sync throw in f becomes a rejection.
- In p.then(f, r), r does not see a throw from f; a catch(r) after it does. A non-function handler lets the value pass; finally(f) calls f with no argument and passes the value on, unless f throws.
- Duck typing is history.** Promise libraries predate ES2015, so a promise adopts any thenable (a callable then), which is read now, called a job later (t in drawing 2). So await { then(r) { r(42) } } is 42; a module exporting then hijacks await import(); with Object.prototype.then set, async () => {} never settles.
- A promise never holds a promise:** then maps and flattens. A 2013 Promises/A+ request for a monadic then was turned down as “typed-language fantasy land”. Promise.resolve(p) returns a native p itself, as await does since ES2019.

Drawing 2 in code, and its history

```

const log = (s) => console.log(s), p = Promise.resolve();
(async () => { await p; log('A'); })();
p.then(() => log('B')).then(() => log('C')).then(() => log('D'));
p.then(() => { log('E'); return p; }).then(() => log('F'));
(async () => { return p; })().then(() => log('G'));
(async () => { return await p; })().then(() => log('H'));
log('S');
// S A B E C H D G F
  
```



Until ES2019 await wrapped every operand in a new promise; tc39/ecma262 #1250 made it call PromiseResolve (above). On by default from V8 7.2.

async functions

- Run synchronously up to the first await; the code after it runs in a later job, when state may have changed. return / a throw settle the returned promise.
- return await p beats return p twice. In a try, return p leaves before p returns: catch never sees it. And V8’s async stack traces follow await only: after return await the caller stays in error.stack (at async outer); after return p or a then chain it is gone.

Rejections nobody handles

- The engine only reports: HostPromiseRejectionTracker(p, "reject") with no handler, "handle" once one comes. “Too late” is the host’s call.
- Browser: after a microtask checkpoint → unhandledrejection (cancelable: preventDefault() keeps it off the console); a later handler → rejectionhandled.
- Node only warned until v15; since then --unhandled-rejections defaults to throw: your 'unhandledRejection' listener, else an uncaught exception.

Combinators are logic

The combinators (drawing 3): all = AND of fulfilments, any = OR, race = first to settle, allSettled = wait for all. Empty input gives the identities: all([]) fulfils with [] at once (AND of nothing is true), any([]) rejects at once with an empty AggregateError (OR of nothing is false), race([]) never settles (no runner, no winner).

Traps

- ! pa = getA(); pb = getB(); await pa; await pb — if pb rejects while await pa waits, it has no handler yet: unhandled (Node: uncaught). Use Promise.all.
- ! new Promise(async (resolve) => ...): a throw inside rejects only the executor’s own, unhandled promise; the outer one stays pending.

Concurrency shapes

```

const a = await getA(); const b = await getB(); // tA + tB
const [a, b] = await Promise.all([getA(), getB()]); // max(tA, tB)
items.forEach(async (x) => { await save(x); }); // NOT awaited
for (const x of items) await save(x); // one at a time
await Promise.all(items.map(x => save(x))); // all at once
async function pool(items, n, work) { // at most n
  const out = []; let next = 0;
  const lane = async () => { while (next < items.length) {
    const i = next++; out[i] = await work(items[i]); } };
  await Promise.all(Array.from({ length: n }, lane));
  return out; } // next++ is safe: no job runs between read and write
const { promise, resolve, reject } = Promise.withResolvers();
ws.addEventListener('open', resolve, { once: true });
ws.addEventListener('error', reject, { once: true });
await promise; // an event as a promise
  
```

One failing lane rejects pool at once, yet the other lanes keep pulling items — give work a signal and abort it in catch.

Cancel with AbortSignal

The language has no cancel (TC39 withdrew cancelable promises on 2016-12-15), so Promise.race([work, timeout]) stops only the waiting. The DOM’s AbortSignal is a flag the work must choose to check.

```

const ac = new AbortController();
const signal = AbortSignal.any([ac.signal, AbortSignal.timeout(5000)]);
const res = await fetch(url, { signal }); // rejects with signal.reason
const sleep = (ms, signal) => new Promise((resolve, reject) => {
  signal?.throwIfAborted(); // a throw here = rejection
  const stop = () => { clearTimeout(t); reject(signal.reason); };
  const t = setTimeout(() => {
    signal?.removeEventListener('abort', stop); resolve(); }, ms);
  signal?.addEventListener('abort', stop, { once: true });
});
  
```

- abort(reason) runs the abort algorithms and the abort event before it returns. The reason travels — AbortError by default, TimeoutError from timeout() — so a catch tells “too slow” from “cancelled”; any() keeps the reason of the signal that fired, and cannot unsubscribe from its inputs: remove your listeners. Node ships the sleep above in node:timers/promises.

Retry with full jitter

```

async function retry(fn, { tries = 5, base = 100, cap = 10_000, signal } = {}) {
  for (let i = 0; i++ < tries) {
    try { return await fn(signal); } catch (e) {
      if (i + 1 >= tries || signal?.aborted) throw e;
      await sleep(Math.random() * Math.min(cap, base * 2 ** i), signal); } }
  }
  
```

Why random: clients that failed together retry together, and plain exponential backoff keeps them in step — bursts and idle gaps. Full jitter, random(0, min(cap, base * 2^{attempt})), spreads them to a near-constant rate: in a 2015 AWS simulation with 100 contending clients it more than halved the calls of un-jittered backoff. Retry only idempotent requests; honour Retry-After on 429 / 503.

Async iteration and disposal

```

async function* pages(url, signal) { // Symbol.asyncIterator
  while (url) {
    const page = await (await fetch(url, { signal })).json();
    yield* page.items; url = page.next; } }
for await (const it of pages('/feed')) if (it.last) break; // -> return()
const all = await Array.fromAsync(pages('/feed')); // ES2026, one by one
  
```

- break / throw in for await calls return(): the generator’s finally runs. Calls to next() queue — one at a time. In an async generator yield v awaits v first.
- for await (x of [p1, p2]) also works, one at a time — a p2 that rejects while p1 is awaited has no handler yet (the first trap).
- await using r = ... (finished, ES2027): at block exit, also on throw, r[Symbol.asyncDispose]() (else Symbol.dispose) is awaited, in reverse order; errors combine into SuppressedError. AsyncDisposableStack holds many.